

Técnico Especialista em Gestão de Redes e Sistemas Informáticos

UC 606 - Relatório de Apresentação do Projeto Final

Plataforma de Autenticação Resiliente e Analisador de Logs



Aluna: Maria Eduarda Girardi

1) Introdução

Este projeto consiste numa plataforma de autenticação resiliente em modo consola, desenvolvida em Python, cujo objetivo é gerir utilizadores e acessos de forma segura, aplicando boas práticas de segurança informática.

Em sistemas de autenticação simples, é comum encontrar problemas como:

- armazenamento inseguro de passwords (em texto simples);
- ausência de registo de tentativas de login;
- vulnerabilidade a ataques de força bruta;
- inexistência de mecanismos de bloqueio de contas ou IPs;
- dificuldade em analisar padrões de acesso malicioso.

Este projeto foi criado para resolver esses problemas, simulando um sistema de autenticação mais robusto, semelhante aos utilizados em aplicações reais.

O sistema permite:

- a criação de utilizadores com passwords protegidas através de hash criptográfico;
- a realização de login com verificação de credenciais;
- o registo de todas as tentativas de acesso num ficheiro de logs;
- a deteção de comportamentos suspeitos, como múltiplas tentativas falhadas;
- a aplicação de bloqueios temporários de contas e bloqueio de IPs através de uma blacklist.
- O projeto está organizado em vários módulos, cada um com uma responsabilidade específica, promovendo uma estrutura clara, segura e fácil de manter.

Funcionamento da Plataforma de Autenticação Resiliente:

O sistema desenvolvido funciona de forma integrada e automática, seguindo uma sequência lógica de segurança.

1. O utilizador tenta autenticar-se no sistema, fornecendo o nome de utilizador, a password e um endereço IP (simulado).
2. Antes de qualquer verificação de credenciais, o sistema consulta a blacklist para verificar se o IP se encontra bloqueado.
3. Caso o IP esteja bloqueado, o acesso é imediatamente recusado.
4. Se o IP não estiver bloqueado, o sistema verifica se a conta do utilizador se encontra temporariamente bloqueada devido a tentativas falhadas anteriores.
5. A password introduzida é validada através de hashing seguro (PBKDF2-HMAC-SHA256 com salt único).
6. O resultado da tentativa de login (sucesso ou falha) é registado no ficheiro de logs.
7. Após falhas de autenticação, o analisador de logs é executado, detetando padrões suspeitos e atualizando automaticamente a blacklist, se necessário.

2) Estrutura do Sistema, Módulos e suas funções

Ficheiro	Papel no Projeto	Funções / Responsabilidades Principais
login_seguro_tk.py	Ficheiro principal	Apresenta o menu da aplicação, permite criar utilizadores, gere o processo de login, coordena os restantes módulos, regista tentativas de acesso e ativa a análise de segurança.
io_utils.py	Gestão de ficheiros input/output	Lê e escreve ficheiros JSON e CSV, cria diretórios automaticamente, garante escrita atómica de dados sensíveis e regista logs de tentativas de login.
parser.py	Leitura e interpretação de logs	Lê o ficheiro CSV de logs, converte cada linha num objeto LogEntry , valida e normaliza os dados (datas, IPs, sucesso/falha).
analytics.py	Análise e segurança	Analisa os logs processados pelo parser, identifica comportamentos suspeitos, gere a blacklist de IPs, verifica bloqueios ativos e gera relatórios de segurança.
users_secure.json	Dados dos utilizadores	Armazena os utilizadores, hashes das passwords, salts, número de falhas e informação de bloqueio temporário das contas.
logs_exemplo.csv	Registo de acessos	Guarda o histórico de todas as tentativas de login, incluindo data/hora, utilizador, IP e resultado da tentativa.
blacklist.json	Lista de IPs bloqueados	Contém os IPs bloqueados pelo sistema, o motivo do bloqueio e o tempo até ao desbloqueio.

3) Testes Realizados:

3.1) Tentativa de login com IP previamente bloqueado

```
=== Plataforma de Autenticação Resiliente ===
1) Criar utilizador
2) Fazer login
3) Analisar logs / atualizar blacklist
0) Sair
Escolhe uma opção: 2
Utilizador: teste4
IP de origem (simulado): 203.0.113.55
❌ Acesso recusado. IP bloqueado (manual_test).
```

3.2) Tentativa de login com utilizador errado

```
=== Plataforma de Autenticação Resiliente ===
1) Criar utilizador
2) Fazer login
3) Analisar logs / atualizar blacklist
0) Sair
Escolhe uma opção: 2
Utilizador: Admiin
IP de origem (simulado): 192.168.1.43
❌ Utilizador não existe.
```

3.3) Tentativa de login com senha errada

```
=== Plataforma de Autenticação Resiliente ===
1) Criar utilizador
2) Fazer login
3) Analisar logs / atualizar blacklist
0) Sair
Escolhe uma opção: 2
Utilizador: admin
IP de origem (simulado): 192.168.1.43
Password:
❌ Credenciais inválidas.
```

3.4) Tentativa de login com sucesso

```
=== Plataforma de Autenticação Resiliente ===
1) Criar utilizador
2) Fazer login
3) Analisar logs / atualizar blacklist
0) Sair
Escolhe uma opção: 2
Utilizador: admin
IP de origem (simulado): 192.168.1.43
Password:
✅ Login com sucesso.
```

3.5) Criar utilizador com senhas que não coincidem

```

=== Plataforma de Autenticação Resiliente ===
1) Criar utilizador
2) Fazer login
3) Analisar logs / atualizar blacklist
0) Sair
Escolhe uma opção: 1
Novo utilizador: teste 5
Password:
Confirmar password:
❌ As passwords não coincidem.

```

3.6) Criar utilizador com senha demasiado curta

```

=== Plataforma de Autenticação Resiliente ===
1) Criar utilizador
2) Fazer login
3) Analisar logs / atualizar blacklist
0) Sair
Escolhe uma opção: 1
Novo utilizador: teste6'
Password:
Confirmar password:
❌ Password demasiado curta (mínimo sugerido: 6).

```

3.4) Analisar logs/relatorio ('fails': 183)

```

=== Plataforma de Autenticação Resiliente ===
1) Criar utilizador
2) Fazer login
3) Analisar logs / atualizar blacklist
0) Sair
Escolhe uma opção: 3
📄 Relatório:
{'generated_at_utc': '2026-01-30T01:29:51.972571+00:00', 'stats': {'total': 253, 'success': 70, 'fails': 183, 'fails_by_ip_top10': [('198.51.100.77', 34), ('203.0.113.10', 13), ('192.168.1.27', 10), ('203.0.113.200', 8), ('192.168.1.45', 7), ('192.168.1.43', 7), ('192.168.1.20', 5), ('192.168.1.17', 5), ('192.168.1.40', 5), ('192.168.1.29', 5)], 'fails_by_user_top10': [('admin', 34), ('madu', 27), ('duda', 16), ('carla', 15), ('ines', 15), ('ana', 11), ('guest', 11), ('tiago', 9), ('sofia', 9), ('bruno', 9)]}, 'new_blocks_applied': [], 'blacklist_count': 2}

```

3.4) Analisar logs/relatorio após mais um login errado ('fails': 184)

```

=== Plataforma de Autenticação Resiliente ===
1) Criar utilizador
2) Fazer login
3) Analisar logs / atualizar blacklist
0) Sair
Escolhe uma opção: 3
📄 Relatório:
{'generated_at_utc': '2026-01-30T01:31:31.310517+00:00', 'stats': {'total': 254, 'success': 70, 'fails': 184, 'fails_by_ip_top10': [('198.51.100.77', 34), ('203.0.113.10', 13), ('192.168.1.27', 10), ('203.0.113.200', 8), ('192.168.1.45', 7), ('192.168.1.43', 7), ('192.168.1.20', 5), ('192.168.1.17', 5), ('192.168.1.40', 5), ('192.168.1.29', 5)], 'fails_by_user_top10': [('admin', 34), ('madu', 27), ('duda', 16), ('carla', 15), ('ines', 15), ('ana', 11), ('guest', 11), ('tiago', 9), ('sofia', 9), ('bruno', 9)]}, 'new_blocks_applied': [], 'blacklist_count': 2}

```

4) Códigos Relevantes:

4.1) O hashing da password (login_seguro.tk.py);

```

#hash seguro
def pbkdf2_hash_password(password: str, salt: bytes, iterations: int = 200_000) -> bytes:
    return hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, iterations)

```

```

salt = os.urandom(16)
iterations = 200_000
pwd_hash = pbkdf2_hash_password(pw1, salt, iterations)

```

```

test = pbkdf2_hash_password(password, salt, iterations)
return hmac.compare_digest(stored, test)

```

Implementação: O hashing da password foi implementado com o algoritmo PBKDF2 com SHA-256, utilizando um salt aleatório e um elevado número de iterações. O hash e o salt são armazenados de forma segura e a verificação da password é feita através da comparação segura dos hashes.

4.2) O registro de logs:

a.1) login_seguro_tk.py

```
def log_attempt(username: str, ip: str, success: bool) -> None:
    ts = _now_utc().isoformat().replace("+00:00", "Z")
    append_csv_row(
        LOGS_PATH,
        headers=LOG_HEADERS,
        row={
            "timestamp": ts,
            "utilizador": username,
            "ip": ip,
            "sucesso": "true" if success else "false",
        },
    )
```

a.2) io_utils.py

```
def append_csv_row(path: str, headers: List[str], row: Dict[str, Any]) -> None:
    ensure_parent_dir(path)
    file_exists = os.path.exists(path)
    needs_header = (not file_exists) or (os.path.getsize(path) == 0)

    with open(path, "a", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=headers)
        if needs_header:
            writer.writeheader()
        writer.writerow(row)
```

Implementação: O registo de logs foi implementado através da escrita de cada tentativa de login num ficheiro CSV, utilizando uma função dedicada que regista a data/hora em UTC, o utilizador, o IP e o resultado da tentativa, permitindo posterior análise de segurança.

4.3) A análise de logs e bloqueio automático:

```
def detect_suspicious_ips(entries: List[LogEntry], now: Optional[datetime] = None) -> List[BlockDecision]:
    """
    Avalia regras e devolve lista de decisões de bloqueio.
    """
```

```
# FUNÇÃO PRINCIPAL:|

def analyse_and_update_blacklist(
    logs_path: str = "logs_exemplo.csv",
    blacklist_path: str = "blacklist.json",
    now: Optional[datetime] = None,
) -> Dict:
    """
    Lê logs, calcula estatísticas, detecta IPs suspeitos e atualiza
    blacklist.json.
    Retorna um relatório (dict).
    """
    now = now or _now_utc()
    entries = try_read_logs_csv(logs_path) or []

    blacklist = load_blacklist(blacklist_path)
```

Implementação: A análise de logs e bloqueio automático foi implementada através da função **detect_suspicious_ips**, que percorre os registos de autenticação, identifica padrões de força bruta com base em janelas temporais e gera decisões de bloqueio. Essas decisões são automaticamente aplicadas na função **analyse_and_update_blacklist**, que atualiza o ficheiro blacklist.json, criando bloqueios temporários ou permanentes conforme as regras definidas.

5) Medidas de Cibersegurança Aplicadas:

a.1) Hashing seguro de passwords:

As passwords não são armazenadas em texto simples.

- Utiliza-se o algoritmo PBKDF2 com SHA-256
- É aplicado um salt aleatório por utilizador
- Usa um número elevado de iterações (200 000)

Isto dificulta ataques de força bruta e uso de rainbow tables.

a.2) Comparação segura de hashes:

Na verificação de login:

- a comparação do hash da password é feita com `hmac.compare_digest`

Evita ataques por timing, que exploram diferenças de tempo na comparação de strings.

a.3) Bloqueio temporário de contas (Lockout):

Após várias tentativas falhadas:

- a conta do utilizador é temporariamente bloqueada
- o tempo de bloqueio aumenta de forma exponencial (backoff)

Esta medida reduz ataques de força bruta direcionados a contas específicas.

a.4) Registo detalhado de logs:

Todas as tentativas de login são registadas:

- data e hora em UTC
- utilizador
- IP de origem
- sucesso ou falha

Os logs permitem auditoria, análise forense e deteção de ataques.

a.5) Análise automática de logs:

O sistema analisa os logs para:

- identificar múltiplas falhas provenientes do mesmo IP
- detetar padrões suspeitos num curto período de tempo

A análise é baseada em dados estruturados (LogEntry).

a.6) Blacklist dinâmica de IPs:

Quando um IP apresenta comportamento suspeito:

- é adicionado automaticamente a uma blacklist
- o bloqueio tem tempo de expiração

IPs bloqueados são impedidos de tentar login antes mesmo da validação da password.

a.7) Escrita segura de dados sensíveis:

Os ficheiros críticos (users_secure.json, blacklist.json) são escritos de forma atómica.

Evita corrupção de dados em caso de falha do programa.

a.8) Normalização e validação de dados:

- Datas são convertidas para UTC
- Valores de sucesso são convertidos para booleanos
- Linhas inválidas de logs são ignoradas

Isto aumenta a robustez do sistema contra dados malformados.

6) Conclusão e Aprendizagens

Aprendi a importância de nunca guardar passwords em texto simples e de utilizar funções de hashing seguras com salt e várias iterações, garantindo uma melhor proteção das credenciais dos utilizadores. Compreendi também como a comparação segura de hashes ajuda a evitar ataques mais subtis, como ataques por timing.

Ao longo do projeto, percebi como o registo de logs é fundamental para monitorizar tentativas de acesso e identificar padrões suspeitos. A análise desses logs permitiu-me entender como é possível detetar ataques de força bruta e aplicar bloqueios automáticos de contas e IPs, reforçando a segurança do sistema.

Aprendi ainda a importância da separação de responsabilidades entre módulos, organizando o código de forma mais clara e fácil de manter. Dividir o projeto em componentes como autenticação, gestão de ficheiros, parser e análise de segurança tornou o desenvolvimento mais estruturado.

Por fim, este projeto ajudou-me a compreender melhor como princípios de cibersegurança podem ser aplicados na prática, mesmo em sistemas simples, aumentando a minha consciência sobre a necessidade de desenvolver aplicações mais seguras desde a sua base.